

UNIVERSITY OF VERONA

DEPARTMENT OF COMPUTER SCIENCE

MASTER'S DEGREE IN DATA SCIENCE

Querying heterogeneous data through
Knowledge Graphs and LLM

Supervisor:

Elisa Quintarelli

Candidate:

Indeewari Balasooriya

VR510479

Co-Supervisor:

Matteo Lissandrini

ACADEMIC YEAR 2024/2025

Abstract

This thesis explores querying heterogeneous data sources using Knowledge Graphs (KGs) and Large Language Models (LLMs). It proposes a polyglot query engine that integrates multi-model data into a unified graph-based semantic layer through Virtual Knowledge Graphs (VKGs), and Ontop for ontology-based data access (OBDA). The work leverages LLMs to translate natural language questions into SPARQL queries to be applied to the VKGs, incorporating SHACL for data validation and exploring structured guidance to optimize query generation. The research bridges natural language interfaces and semantic technologies, contributing both practical tools and theoretical insights.

Contents

1	Introduction	3
2	Background	7
2.1	RDF	7
2.2	SPARQL	10
2.3	Knowledge Graphs	11
2.4	Shapes	14
2.5	M-Schema	17
2.6	LLMs	17
3	Related Work	19
4	Case Study	23
5	Translating NL to SparQL with LLMs	36
5.1	M-Schema vs Shapes	36
5.2	Optimizing Prompt Structure for Lightweight LLMs: Llama- 2-7B (Quantized) Experiments	46
6	Conclusions	60

Chapter 1

Introduction

In today’s digital landscape, companies, organizations, and individuals frequently handle vast amounts of data originating from diverse sources, including structured and semi-structured databases, web pages, social media platforms, and unstructured files. These datasets often exist in various formats and require different processing techniques and query languages for access and analysis. This heterogeneity poses a significant challenge when attempting to consolidate data into a coherent and unified view. To overcome this, *data integration* plays a fundamental role.

Data integration is a crucial process in modern data management that involves combining data from multiple, disparate sources into a single, consistent dataset. This unified view is essential for enabling comprehensive analysis, supporting informed decision-making, and fostering the development of intelligent systems. Without effective data integration, key information and semantic relationships may remain obscured, thereby limiting the potential of knowledge discovery and analytic insights.

In this regard, **Knowledge Graphs (KGs)** have become a powerful paradigm for data integration. What makes KGs powerful is that they offer a structured and semantically enriched representation of information that allows the linking and querying of heterogeneous data sources. KGs do not only store data, but also capture the meaning (semantics) of data.

Following the paradigm of KGs, a **Virtual Knowledge Graph (VKG)** creates a virtual layer that connects original data sources while keeping the data at its original location. This does not build a KG by physically copying all data (called materialization), which is source-expensive. Following the **Global-as-View approach**, VKGs work as a global schema of all the available data sources, which gives the user the feeling that all data is available in one source. This approach enables the querying of diverse data models

using a unified query language, typically **SPARQL**. This abstracts away the complexity of underlying heterogeneous data models. Thus, Virtual Knowledge Graph allows users to access and query different data sources without worrying about the specific formats or structures of each individual model.

However, for casual users and even for some developers, accessing data through VKGs requires technical knowledge to write SPARQL queries. This creates a barrier to expanding the use of KGs and VKGs. Recent advances in Artificial Intelligence (AI) and Large Language Models (LLMs) such as Llama, GPT-4, and others have opened exciting new possibilities. LLMs are capable of understanding and generating human-like language, and they can be trained to translate natural language questions (like "Show me all the events in Verona") into formal SPARQL queries. This work experiments with the possibility of seamlessly translating **natural language** queries into formal **SPARQL** queries using **LLMs**. This is useful to automate the interaction with Virtual Knowledge Graphs, enabling natural language interfaces for querying complex, integrated data sets. It fills the gap between users and KGs/VKGs.

This thesis is conducted within the scope of **DataGEMS**(Data-driven Analytics and Tools for Advanced Graphed-based Entity Management Systems), which focuses on building a polyglot engine to enhance the field of data integration. A polyglot engine is a system that enables managing data from different formats and allows users to query them in a unified way. The existing data management and discovery systems face some limitations: they are based on basic keyword searches, datasets lacking metadata, inability to search through different data sources, and most often can merely search simple datasets (like spreadsheets). The main goal of DataGEMS is to make data more FAIR (Findable, Accessible, Interoperable, and Reusable). Traditional data management systems basically support storing and querying structured data. And data discovery (finding data) and data exploration (understanding what is inside the data) are handled differently. DataGEMS focuses on moving further with a **MoMa (Unified Model for Data Management and Discovery)**: combine data discover and exploration allowing users to move directly from discovering datasets to analyze them and use machine learning workflows to queries and **DSDS (Dataset Management and Discovery System)**: build a federated data lake—a shared platform to access datasets without worrying about complex integration.

Contributions The **main aims** of this thesis extend to,

- design a polyglot engine within the DataGEMS frameworks that en-

sures data integration through Virtual Knowledge Graphs and

- leverage large language models in translating natural language questions to structured queries like SPARQL.

This includes:

- conducting an extensive study of the state-of-the-art applications of Knowledge Graphs in data integration.
- using the open source framework OnTop as a polyglot engine that supports multiple data models and query languages using a Virtual Knowledge Graph-based approach.
- investigating the application of Large Language Models for translating user queries in natural language directly into SPARQL.

Structure of the Thesis The Thesis is organized in the following Chapters.

- Chapter 2 provides *foundational knowledge*, discussing RDF as a standard model for data exchange, SPARQL as a query language for accessing RDF data, the idea and use of Knowledge Graphs in data integration, an overview of Large Language Models, their advancements and ability to translate human language to SPARQL, an introduction to M-schema, a schema-based approach that describes the structure of the data sources, Shapes that are used to define constraints over RDF data basically to validate them and Virtual Knowledge Graphs that allows querying different data sources, without physically moving or transforming them.
- Chapter 3 includes *related work* that identifies the gaps addressed by the thesis by reviewing existing research on polyglot systems and data integration using Knowledge Graphs. Moreover, existing methods for translating Natural Language (NL) to SPARQL queries, including how different models, particularly LLMs, have been applied to this task.
- Chapter 4 includes a detailed *case study* describing the database and resources used in the experiments: ER diagram of the database, relevant TTL, mapping, and M-schema files.
- Chapter 6 focuses on how *LLMs can be used to translate natural language queries into SPARQL*, especially focusing on the difference

in performance of the model when guiding the translation with M-schema-based and Shapes-based approaches. SparQL queries are then executed on the available dataset through the OnTop framework.

- Finally, Chapter 7 concludes the thesis.

Chapter 2

Background

This chapter explains the key concepts and technologies that are useful to understand the rest of the thesis work. This includes the concepts of RDF and KGs that aid in organizing data semantically, VKGs: enhanced steps to KGs to access data without physically copying, Shapes that provide constraints over RDF data, M-schema as an approach to structure data, and lastly LLMs basically to understand and process natural language.

2.1 RDF

The Resource Description Framework (RDF) is a foundational model for representing and allowing data exchange on the Web. It is a standard of the W3C (World Wide Web Consortium) for exchanging graphs. The digital age continues to generate enormous volumes of heterogeneous data, ranging from structured databases to unstructured media such as images, videos, and text, as a result creating an increasing need for a flexible and standardized format to enable smooth integration, interoperability, and semantic understanding. RDF addresses this challenge by offering a graph-based data model that organizes information in the form of triples—subject(an IRI or blank node), predicate(an IRI), and object(an IRI, a blank node, or a literal). This enables the representation of both raw facts and complex relationships between entities. The IRIs (Internationalized Resource Identifiers), uniquely identify entities and relationships; literals, store data values like text, numbers, and dates; and blank nodes, represent unnamed entities. The subject represents an entity, the predicate defines a relationship or property, and the object provides the value or another entity connected to the subject. For example, the statement “*The event La Grande Sfida takes place at Piazza Bra - Verona*” can be represented as the triple (*La Grande Sfida*, *eventat*, *Piazza*

Bra - Verona). This simple yet powerful model enables data to be linked across multiple sources, thus enhancing semantic interoperability between systems. RDF is used to model KGs. As seen above, since RDF’s graph structure allows greater expressivity than traditional tabular databases, it is an effective tool for knowledge representation, linked data integration, and semantic reasoning [1].

RDF can be serialized in a number of formats to accommodate different use cases. RDF/XML was historically the standard format, but is frequently regarded as verbose. JSON-LD is particularly useful for web applications, as it integrates RDF data into JSON forms that are easier to process by contemporary APIs [2]. Turtle (TTL) provides a more human-readable syntax, allowing RDF data to be written in a clear and concise way. Other formats, such as N-Triples, provide a line-based syntax that helps efficient storage and transmission. With the help of these different serializations, RDF can be used in a wide range of applications while maintaining data interoperability and machine readability.

The RDF Schema (RDFS) and the Web Ontology Language (OWL) define a formal structure for RDF data while enhancing the expressive power of RDF. RDFS offers a lightweight schema mechanism that supports the creation of classes, properties, and constraints (simple rules). For instance, an entity classified as an Event can have properties such as name, date, and start time, where each property has an expected type. OWL expands this capability by introducing more advanced reasoning mechanisms, such as equivalence, subclass relationships, and property restrictions. With the help of these semantic extensions, automated reasoning systems can infer new knowledge from existing data. This makes RDF a powerful tool for knowledge representation and logical inference.

Another key advantage of RDF is its ability to facilitate Linked Data. RDF enables data from different sources to be connected in a meaningful way by using Uniform Resource Identifiers (URIs). Moreover, as RDF supports the use of standardized vocabularies such as FOAF (Friend of a Friend), Schema.org, and DBpedia, knowledge can be shared, reused, and integrated across domains. For example, consider an RDF dataset describing research papers. This dataset can be linked to external author profiles on ORCID, citation data in DBLP, and related topics in Wikidata, creating a rich network of interconnected knowledge.

To illustrate these concepts, consider the RDF representation of La Grande Sfida—an international event of culture, sports, arts, and dialogue held in Verona. The following N-Triples representation describes this event while also linking to external sources to demonstrate RDF’s ability to con-

nect distributed knowledge.

```
<https://example.org/47602> <https://example.org/
  eventname_it> "20^ La Grande Sfida"^^<http://www.
  w3.org/2001/XMLSchema#string> .
<https://example.org/47602> <https://example.org/
  eventat> "Piazza-Bra-Verona"^^<http://www.w3.org
  /2001/XMLSchema#string> .
<https://example.org/47602> <http://www.w3.org
  /1999/02/22-rdf-syntax-ns#type> <https://example.
  org/Event> .
<https://example.org/Event> <http://www.w3.org
  /2002/07/owl#equivalentClass> <http://www.
  wikidata.org/entity#Q1656682> .
```

where IRIs are enclosed in angle brackets `< >`. Here, La Grande Sfida is linked to the Event entity in Wikidata, via the predicate

`<http://www.w3.org/2002/07/owl#equivalentClass>`. This demonstrates how RDF can integrate external knowledge sources such as Wikidata, creating a semantic web of interlinked information.

Using Turtle, the previous N-Triples example can be rewritten as:

```
BASE <https://example.org/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-
  -ns#> .
PREFIX owl: <http://www.w3.org/2002/07/owl#> .
PREFIX wd: <http://www.wikidata.org/entity#> .

<47602> <eventname_it> "20^La Grande Sfida"^^xsd:
  string ;
      <eventat> "Piazza-Bra-Verona"^^xsd:
        string ;
      rdf:type <Event> .
<Event> owl:equivalentClass wd:Q1656682 .
```

This version reduces repetition, improving readability without changing the underlying meaning. The use of prefixes (rdfs:, xsd:) and relative IRIs (`<eventat>`) makes the data easier to manage and reuse across different datasets.

2.2 SPARQL

SPARQL (SPARQL Protocol and RDF Query Language) is the W3C standardized query language for RDF data [3]. Initially introduced as a standard in 2008 and in 2013, updated to SPARQL 1.1, which is mainly supported by graph databases. The schema-free, flexible structure helps SPARQL to uncover complex relationships and retrieve information by performing complex joins among different databases within a single query, in contrast to traditional query languages, such as SQL, which work on structured databases with fixed schemas.

When considering major components of SPARQL specification, other than as a query language, firstly, it supports multiple output formats such as XML, JSON, CSV, and TSV, thus providing flexibility to retrieve results in the format that suits to each use case. Secondly, it works as an update language that enables the modification of RDF data through inserting, updating, and deleting triples. Thirdly, it facilitates interaction with remote SPARQL services using standardized web-based protocols. Finally, SPARQL includes a service description vocabulary that provides metadata about SPARQL endpoints. This is helpful to understand what kind of queries and datasets are supported.

At a high level, the structure of a SPARQL query consists of the pattern matching part, solution modifiers, and the output [4]. In the **pattern matching part**, the query searches for specific patterns in the RDF data using features like *optional patterns*, *unions*, *nesting*, and *filters*. Once the result from pattern matching is ready, the **solution modifiers** further allows to refine them using operations like *projection (choose specific variables)*, *DISTINCT (ensure uniqueness)*, *GROUP BY (grouping)*, *ORDER BY (ordering)*, and *LIMIT/OFFSET (limiting the number of results)*. Finally, the **output** of a SPARQL query can be several types such as *yes/no (ASK queries)*, *selections of values of the variables that match the pattern (SELECT queries)*, *constructions of new RDF triples (CONSTRUCT queries)*, or *resource descriptions (DESCRIBE queries)*. At the structural level, a SPARQL query typically consists of: a **prologue**, at the very beginning defining the PREFIX and BASE IRIs, a **SELECT clause** specifying variables to be included in a result which can be further enhanced including functions like *COUNT*, *SUM*, *AVG*, *MIN*, and *MAX*, a **WHERE clause** providing the condition that which pattern should be matched, and **solution modifiers** as discussed.

For example the following query retrieves the name, date, location and image of events planned to held after a specific date, using a basic pattern

matching (SELECT query), FILTER to show only events after *2015-12-12*, OPTIONAL to include image only if it exists in the dataset and the solution modifier ORDER BY to sort results by ascending order of the date. The prologue part consists of the PREFIX declaration.

```

PREFIX ex: <http://example.org/>

SELECT ?eventName ?date ?location ?image
WHERE {
  ?event a ex:Event ;
        ex:eventname_it ?eventName ;
        ex:eventat ?location ;
        ex:eventon ?calendar .
  ?calendar ex:day ?date .

  OPTIONAL { ?event ex:eventimage_url ?image }
  FILTER(?date >= "2015-12-12"^^xsd:date)
}
ORDER BY ASC(?date)

```

The results of the SPARQL query can be seen in Figure 2.1.

Execution time: 1.37s. Solution mappings returned: 9927.

SPARQL results	SQL translation			
eventName	date	location	image	
"Le Meraviglie del 2000 - Opere della C...	"2015-12-12"^^xsd:date	<http://example.org/location/49029_14500387...	"http://portale.comune.verona.it/media/_ComVR...	
"Tamara de Lempicka"^^xsd:string	"2015-12-12"^^xsd:date	<http://example.org/location/48944_14542723...	"http://portale.comune.verona.it/media/_ComVR...	
"Pippo non lo sa (nel 1940 l'Italia canta l'...	"2015-12-12"^^xsd:date	<http://example.org/location/48574_14500387...	"http://portale.comune.verona.it/media/_ComVR...	
"Palafitte. Un viaggio nel passato per ali...	"2015-12-12"^^xsd:date	<http://example.org/location/48413_14663647...	"http://portale.comune.verona.it/media/_ComVR...	
"Aspettando Santa Lucia"^^xsd:string	"2015-12-12"^^xsd:date	<http://example.org/location/49886_14476196...	"http://portale.comune.verona.it/media/_ComVR...	
"Dario Fo dipinge Maria Callas"^^xsd:string	"2015-12-12"^^xsd:date	<http://example.org/location/49235_14542723...	"http://portale.comune.verona.it/media/_ComVR...	
"Il tesoro misterioso" di Herbert Hamak a...	"2015-12-12"^^xsd:date	<http://example.org/location/49171_14500388...	"http://portale.comune.verona.it/media/_ComVR...	
"Scandalo"^^xsd:string	"2015-12-12"^^xsd:date	<http://example.org/location/49059_14500387...	"http://portale.comune.verona.it/media/_ComVR...	

Figure 2.1: SPARQL Query Results

2.3 Knowledge Graphs

A knowledge graph is a way to organize and represent data using nodes (entities) and edges (relationships). It creates a meaningful, interconnected network of information. A KG is like a mind map for data. Instead of keeping pieces of information separate, it connects them all. Unlike traditional databases that store data in isolated tables or documents, KGs emphasize meaningful relationships between data points. This enables contextual understanding, reasoning, and discovery. KGs are extensively used in data integration, search engines, recommendation systems, and artificial intelligence applications.

Knowledge Graphs are built using the Resource Description Framework (RDF) along with ontologies. As discussed, they outline entities, properties, and relationships to structure information within a specific domain or across multiple domains. Each entity in a KG is uniquely identified using Uniform Resource Identifiers (URIs), as previously stated, guaranteeing global interoperability and linking of data across different sources. For example, in the tourism-based KG considered in this work, an event is represented as an RDF class, adhering to established ontology standards. The following TTL snippet demonstrates how an event entity is defined:

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix schema: <https://schema.org/> .

ex:Event rdf:type owl:Class ;
         owl:equivalentClass wd:Q1656682 ;
         rdfs:label "Event" .
```

Here, *ex:Event* is defined as an OWL class, meaning it represents a general concept of an event. Additionally, it is linked to *Wikidata's event class* (*wd:Q1656682*), ensuring interoperability with external knowledge bases. The *rdfs:label "Event"* provides a human-readable label for the class. By structuring event data this way, **events** described in different sources can be linked to this entity.

One of the key strengths of KGs is their ability to integrate heterogeneous data sources. Traditional data integration consists of connecting data from various sources such as relational and NoSQL databases, document stores, APIs, and structured files. It involves rule creation to determine how the data sources relate to one another. These rules stem from the predefined schemas, or in more simplistic terms, the structure of the data is set up in advance. KGs, in contrast, provide a more flexible approach as data from different formats and models do not have to adhere to a predefined structure having a schema. This facilitates easier integration of diverse data sources. For instance, in this work, it integrates data from heterogeneous sources, including a **PostgreSQL database and CSV files**. The PostgreSQL database contains structured data about *points of interest (POIs) in Verona, including event details and locations*, while *tourist-related data*,

such as individual tourist information and the POIs they visited, is stored in CSV files. Additionally, *weather data* is also stored in CSV format. By using the flexibility of KGs, this data is integrated into a unified graph model that connects the different data sources through meaningful relationships, such as linking tourists to the POIs they visited or linking events with specific locations and times.

To maintain semantic consistency, KGs often utilize ontologies, which usually outline the hierarchical relationships between concepts. Ontologies, most famously written in OWL (Web Ontology Language), compatible with RDF graphs, assist reasoning engines to infer new relationships, identify inconsistencies, and classify entities based on their properties. For example, when considering this tourism-related Knowledge Graph, the entity **ex:Art** represents *points of interest in Verona*, such as *Casa di Giulietta*. However, **Art** can also be considered a **TouristAttraction** in some contexts. Thus, to build this equivalence, the ontology uses the OWL property **owl:sameAs** to elaborate that *ex:Art is equivalent to schema:TouristAttraction* from the Schema.org vocabulary. This enables the inferring: *ex:Art* can be treated as a **TouristAttraction** even if it is originally categorized differently.

Knowledge Graphs can be created manually, automatically extracted, or constructed using machine learning techniques. Manually created KGs, such as Wikidata and DBpedia, are built from organized inputs from specialists and crowdsourced information. Automated KGs, on the other hand, use Natural Language Processing (NLP), entity recognition, and relation extraction to derive knowledge from unstructured text, web pages, and academic publications. Machine learning-based approaches, such as Graph Neural Networks (GNNs) and Knowledge Graph Embeddings, facilitate the prediction of missing links, clustering of similar entities, and graph-based recommendation systems.

While traditional KGs, are physically transforming and storing data in a graphical structure as discussed, **Virtual Knowledge Graphs** keep data in its original form (like relational databases or CSV files) and virtualize them as a graph. Users can usually query from a KG, typically using SPARQL. VKGs combine the ideas of *virtualization*: users are not exposed to actual dataset, instead interact with a global schema (a high level domain representation) where the integrated view is virtual not materialized, *graph structuring*: represents in graphical structure, and *knowledge embedding*: graph with domain knowledge (hierarchies, properties and domain/range restrictions). VKGs are often referred to as Ontology-Based Data Access (OBDA), which captures the domain knowledge in ontologies usually expressed in Web Ontology Language (OWL2). OWL2 is based on description logics (DLs). In

OBDA, the domain ontology (target) is mapped to data sources (source) [5]. For instance, in the following example, the *State* table in a relational database is represented by a table *d_art_e_stato*, with columns: *code* and *name*. In the target ontology, each row from this table is mapped to an instance of the class *ex:State*, where the *code* is assigned to the property *ex:statecode* and the *name* is assigned to the property *ex:statename*.

target	<code>ex:arteventStatus/{code} a ex:State</code>
	<code>; ex:statecode {code}^^xsd:string ; ex:statename</code>
	<code>{name}^^xsd:string .</code>
source	<code>SELECT "code", "name"</code>
	<code>FROM d_art_e_stato</code>

Through this mapping, the RDF graph is virtualized when queries. When considering VKGs, there are technologies to create the mappings to virtualize data into a KG. **Ontop** is such an open-source OBDA platform that allows querying relational databases as virtual RDF graphs [6]. When a user queries, it internally translates SPARQL queries into SQL queries. VKGs are beneficial as they require less storage and provide real-time access to up-to-date data.

2.4 Shapes

One challenging issue still not mentioned is managing the quality of the data store in KGs and ensuring efficient query processing. To address these challenges, some state-of-the-art languages like Shapes Constraint Language (SHACL) and the Shapes Expression Language (ShEx) were recommended by W3C and the ShEx Community Group, respectively. These languages define validating *shapes* for KGs. Even though the primary focus of these shapes is to validate data, they also have the capability to optimize the SPARQL queries.

In SHACL validation, shapes define constraints that RDF data must follow. These shapes are structured in a **shape graph**, that contains SHACL rules indicating how RDF nodes should be validated [7]. The actual RDF data to be checked against these constraints is stored in **data graph**. Among many possible constraints, a shape can typically include conditions like property constraints (e.g., an *event* must have a name), value constraints (e.g., *start time* must be in "00:00" format), and cardinality restrictions (e.g., an *event* must have exactly one *location*). These rules help to detect inconsistencies, missing values, or incorrect data types.

For instance, the following SHACL shapes define constraints for **Event entity** *ex:Event* and **Calendar entity** *ex:Calendar*. Both **ex:CalendarShape** are *OWL named individuals* and *SHACL node shapes* (meaning they describe constraints for RDF nodes of type *ex:Event* and *ex:Calendar*).

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
ex:EventShape rdf:type owl:NamedIndividual ,
               sh:NodeShape ;
               sh:property [ sh:datatype xsd:string ;
                             sh:maxCount 1 ;
                             sh:minCount 1 ;
                             sh:path ex:eventclassid
                                   ;      <http://www.w3.org/ns/shacl/dash#
                                   uniqueValueForClass>
                                   ex:Event
                             ] ,
               [ sh:class ex:Location ;
                 sh:minCount 1 ;
                 sh:path ex:eventat
               ] ,
               [ sh:datatype xsd:string ;
                 sh:minCount 1 ;
                 sh:path ex:eventname_it
               ] ;
               sh:targetClass ex:Event .

ex:CalendarShape rdf:type owl:NamedIndividual ,
                  sh:NodeShape ;
                  sh:property [ sh:datatype xsd:
                                string ;
                                sh:message "End time
                                             must be in HH:MM
                                             format (00:00 -
                                             23:59)." ;
                                sh:path ex:end_time ;
```

```

        sh:pattern
          "^(?:[01]\\d
            |2[0-3]):[0-5]\\d$"
        ] ;
    sh:targetClass ex:Calendar .

```

Within these node shapes, multiple **property shapes** are defined (*sh:path* specifies to which property the constraint is defined): the property *ex:eventclassid* must be a string, must occur exactly once, and must uniquely define an event using `<http://www.w3.org/ns/shacl/dash#uniqueValueForClass>`. The other property shape constraints that an event must be linked to at least one *ex:Location* using the *ex:eventat* property. This ensures that each event has an associated location. Additionally, the *ex:eventname_it* property ensures that every event has a name and must be a string. In *ex:CalendarShape* node shape, the property shape for *ex:end_time* ensures that the value is a string matching the pattern `"^(?:[01]\\d|2[0-3]):[0-5]\\d$"`. This guarantees that the end time is in 24-hour time format (HH:MM). Moreover, the validation message states that the end time must follow this format, helping to correct errors if entered in the wrong format. For instance, if the end time is entered like **13:60** (Minutes cannot be 60) or **7:45** (Should be 07:45): invalid.

Other than validation, as mentioned, SHACL can be utilized to guide the optimization and rewriting of SPARQL queries [8]. Practically, this means that if the RDF graph conforms to the constraints, the unnecessary or redundant query patterns can be removed without affecting the final result. For example, consider the scenario where the SHACL shape guarantees that an *event* must have exactly one *location*. The query was originally written as,

```

PREFIX ex: <http://example.org/>
SELECT ?event ?location WHERE {
    ?event a ex:Event .
    OPTIONAL { ?event ex:eventat ?location }
}

```

can be optimized to

```

PREFIX ex: <http://example.org/>
SELECT ?event ?location WHERE {
    ?event a ex:Event ;
        ex:eventat ?location .
}

```

In this case, the SPARQL query using the OPTIONAL pattern to retrieve location can be simplified directly to a join. This simplifies the query as well as improves the performance.

2.5 M-Schema

M-schema is a semi-structured schema representation designed to enhance LLMs’ understanding of database structures. It provides a hierarchical representation of the database, including detailed metadata such as **table description, table relationships (foreign keys) and column details: name, primary key identifiers, data types, and example values**. The traditional schema representations, like DDL (Data Definition Language) schemas, lack detailed descriptions about the tables and columns. With this detailed schema, M-schema provides a better awareness of the context, and as a result, improves the query accuracy when translating natural language (NL) to structured queries. Originally introduced in **XiYan-SQL**, M-schema was developed to improve natural language to SQL query generation. However, its structured metadata approach is equally beneficial for NL-to-SPARQL translation, as it enables LLMs to generate semantically correct and optimized queries.[9]

2.6 LLMs

Large Language Models (LLMs) are Neural network-based models that are pre-trained on vast amounts of textual data to process and generate human-like text [10]. These models, such as OpenAI’s GPT, Google’s BERT, and Meta’s Llama are built using deep learning techniques, specifically transformer architectures. The ability of these models to perform a variety of language-related tasks like analyzing, generating, and translating text enhances **Natural Language Processing (NLP)** applications.

Pre-trained LLMs are base models, meaning they are more general. Through *fine-tuning*, these base models can be changed to assistant models that are more domain-specific. This is done by further training them on specialized datasets. It enhances model performance in targeted areas. Moreover, *prompt engineering*, a key technique to improve the performance of LLMs, can be achieved by carefully designing prompts that guide the model’s reasoning process, more accurate and context-aware outputs can be obtained.

However, LLMs are considered as *black-box models*: they do not inherit

explicit reasoning mechanisms and usually face issues in capturing factual knowledge reliably [11]. In contrast, KGs are structured knowledge models, as discussed, that explicitly store and organize factual knowledge. Hence, common LLM issues like *hallucinations* (generating incorrect or misleading information) can be avoided by integrating LLMs with KGs. Meanwhile, LLMs can assist with the challenge of creating and maintaining KGs. For instance, by using techniques like named entity recognition (NER) and relation extraction, LLMs facilitate the generation of domain-specific ontologies [12]. Among these bidirectional benefits, **natural language to SPARQL translation** is crucial [13]. Not all users are familiar with SPARQL or structured query languages. As a solution, NL-based querying can be used to access data stored in KGs. LLMs can facilitate querying KGs by automatically translating natural language questions into SPARQL queries.

Chapter 3

Related Work

This chapter focuses on earlier related research works, mainly considering two areas: polyglot systems and translating natural language queries to SPARQL.

Polyglot systems To tackle the problem of handling different databases at once, initially, concepts like federated database systems and multi-database systems were introduced. As a recent advancement to these previous concepts, the concept of polystores emerged: it refers to the systems that integrate different types of databases, like relational, graph, and NoSQL, allowing them to work together. These systems provide all the functionalities of relevant databases, allow distributed transactions across different databases even though they do not follow the same transaction model, and support different query notations.

[14] proposes ScalarDB that facilitates global transactions across different databases, mainly focusing on database-agnosticism (can work with any database), accuracy with serializability, good performance, scalability, and availability. ScalarDB uses a single-level transaction management (SingleTM) approach, which works at the global level, unlike multi-level transaction management (MultiTM) that depends on each database’s own system. This further extends the SingleTM approach with the Cherry Garcia protocol to achieve database-agnosticism and serializability. The architecture includes a user interface layer, a transaction manager, a database abstraction layer defining a common data model that works across databases, and a shim layer to handle specific details related to the database, translation, and general operations to specific queries.

[15] presents ESTOCADA, a system that leverages polystore architectures by highlighting two existing limitations of them: not using redundant

data (same data stored in different systems can be used more efficiently from one) and materialized views (already executed query results) across different databases. By avoiding these limitations, ESTOCADA allows cross-model view-based query rewriting using database principles like Local-As-View (LAV) mappings and materialized views. This transparently selects the best combination of views and stores to enhance query performance across heterogeneous data models.

[16] introduces the SIB Framework (Semantic Integration at Bosch) to tackle the problem of handling heterogeneous data from different machines to do proper manufacturing quality analysis. This applies semantic data integration using the Virtual Knowledge Graph (VKG) approach. First develop an ontology for the Surface Mounting Process (SMT) and link it to Bosch manufacturing data using semantic mappings. Then the framework enables product analysis using SPARQL queries. In this process, data in JSON files is loaded into PostgreSQL and linked to the ontology using Ontop, a VKG engine.

In [17], to handle the issues in data discovery, KGLac is introduced, which creates a knowledge graph, called GLac. This grabs metadata and semantics of datasets using profiling techniques and machine learning-based representation learning. The graph allows semantic data discovery through SPARQL queries and embedding similarity. Data Profiler processes the large datasets and extracts metadata using PySpark DataFrame. A GLac Builder constructs the graph using embeddings (Content-based embeddings using deep learning models and Schema-based embeddings using word embedding techniques), where nodes are datasets, tables, columns (identified via URIs), and edges are similarity relationships (for instance, content similarity, schema similarity), defined using the Lac Ontology. SPARQL and SPARQL-star support for querying relationships and ML tools such as Jupyter Notebooks facilitate downstream data science tasks.

From NL to SPARQL Knowledge Graph Query Answering (KGQA) systems translate NL questions to graph query languages like SPARQL. Most of these systems use Deep Learning approaches, especially LLMs.

[18] introduces Auto-KGQAGPT, a domain-independent framework that performs natural language questions to SPARQL translation using LLMs. Through the preliminary experiments evaluating ChatGPT’s ability to either directly answer NL questions or translate them into SPARQL using various KG inputs (T-Box, A-Box, or both), the authors demonstrate that considering both the T-box (ontology) and A-box (instance data), improves performance of text-to-SPARQL translation. Auto-KGQAGPT works by

selecting smaller relevant KG fragments of both T-Box and A-Box, providing them to the LLM, and selecting the best-performing query from the generated multiple SPARQL queries. This reduces the number of tokens provided to the LLM while maintaining accuracy.

SPARQLGEN is a one-shot approach that leverages large language models with heterogeneous contextual elements in a single prompt [19]. The prompt includes the natural language question, the relevant RDF subgraph, and a correct example query for a different question. The selection of subgraphs to include in the prompt is done by analyzing the correct SPARQL queries (ground truth) for each question. The process involves breaking down the SPARQL query into basic graph patterns (BGPs), replacing variables with their actual values obtained by running the query, and projecting all the involved variables. This builds a smaller, focused subgraph that contains only the information needed to answer the question. This process ensures that the subgraph is minimal but still relevant. The results highlight that including both the relevant subgraph and an example query in the prompt helps improve the quality of the generated SPARQL queries. The experiment used GPT-3 on 3 different question answering datasets: QALD-9, QALD-10, and BESTIARY (a dataset built by the authors themselves), showing that while SPARQLGEN performs strongly on known datasets like QALD-9, its generalization to newer datasets such as QALD-10 and unseen domains like the BESTIARY dataset remains limited.

[20] proposes an approach to extend the Auto-KGQA framework by integrating Retrieval-Augmented Generation (RAG) and few-shot learning. This approach improves query accuracy by retrieving minimal, relevant KG subgraphs and including LLM prompts with extracted query examples. Based on the evaluation, this leads to a significant improvement in zero-shot performance on the SciQA benchmark. Additionally, it proposes an automated method for extracting a minimal T-Box from schemaless knowledge graphs. These contributions highlight how combining semantic retrieval with prompt engineering improves the performance of LLM-based QA systems.

[21] introduces a Retrieval-Augmented Generation (RAG)-based system for generating federated SPARQL queries from bioinformatics knowledge graphs (KGs). This system enhances the LLMs using structured metadata, including VoID (Vocabulary of Interlinked Datasets) descriptions and example queries. This stores metadata and query examples in an embedding space vector database. The retrieval module selects relevant metadata and examples based on user input, which are then fed into the LLM to guide query generation. To ensure the validity of generated queries, the system

includes a post-generation validation and correction layer. This is based on Shapes Expressions (ShEx) schemas that are semi-automatically derived from the contents of SPARQL endpoints, providing constraints that help filter or adjust incorrect query patterns. The limitation is that the system occasionally hallucinates entity identifiers not present in the data and relies on inefficient string comparisons in SPARQL.

Chapter 4

Case Study

In this Thesis, we use as a pilot of DataGEMS, a real-world dataset that captures tourist visits to Points of Interest (PoIs) in Verona, a city in Northern Italy. This dataset comprises approximately 2.1 million records collected over six years (2014-2019), covering approximately 500,000 tourists and nine distinct PoIs (for example, Arena, Juliet’s House, . . .). Each record includes an anonymized user identifier, the PoI identifier and category, and the visit timestamp. In addition, the dataset has been enriched with contextual information, such as weather conditions and semantic and temporal details, including holiday occurrences. These features could be expanded to include data on local tourist events or spatial relationships, such as nearby PoIs.

A Point of Interest (PoI) is a specific location defined by its geographical coordinates, representing a place of potential interest. This may include tourist attractions, stores, and more. PoIs can include various data such as name, address, and other details such as phone numbers and opening hours, which can also be linked to images. The ER diagram in Figure 4.1 represents the structure of the database, including entities such as Tourists(VERONA_CARD), Points of Interest (ART), Weather Conditions (WEATHER_PROCESSED), LOCATION, CATEGORY, LANG, TYPE, TOUR, CALENDAR, and EVENTS. The database schema is designed to support queries related to tourist behaviors and external factors that may influence their visits.

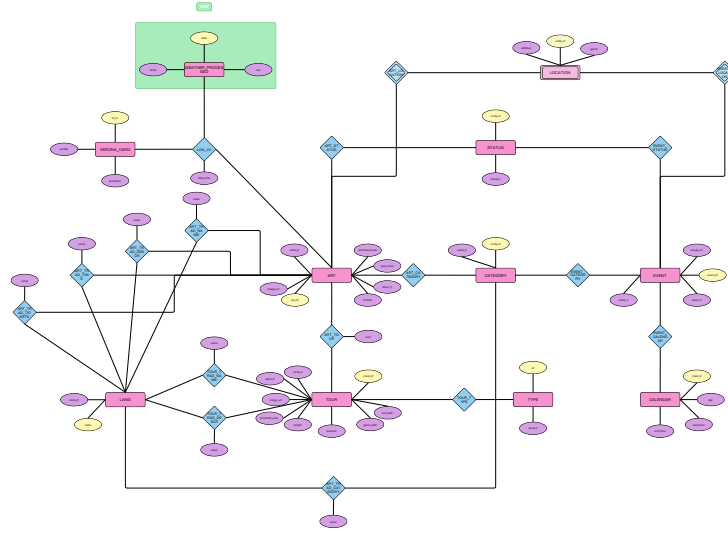


Figure 4.1: Entity-Relationship Diagram of the Database

The database is composed of two major parts: First, *Weather Data* (*CSV format*). The weather data, which includes daily weather conditions for the study period, is stored in a CSV file. This data is highlighted in green in the ER diagram. Each record in the weather dataset corresponds to a specific date and weather details like rain and temperature. Second, *PostgreSQL Database*. The rest of the database is implemented in PostgreSQL, containing the primary tables.

Specifying a TTL file In order to integrate the different portions of the tourism dataset through a VKG, it is first necessary to specify a TTL file that includes the tables and relations from PostgreSQL, except for the VERONA_CARD data. To specify important cardinality constraints, relevant Shapes are also included in the same TTL file.

```
@prefix ex: <http://example.org/> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix wd: <http://www.wikidata.org/entity/> .
@prefix geo: <http://www.opengis.net/ont/geosparql#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix obda: <https://w3id.org/obda/vocabulary#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix schema: <https://schema.org/> .
@prefix geonames: <http://www.geonames.org/ontology#> .
@base <http://www.w3.org/2002/07/owl#> .
```

```

[ rdf:type owl:Ontology
] .

#####
#      Annotation properties
#####

###  http://www.w3.org/ns/shacl#class
sh:class rdf:type owl:AnnotationProperty .

###  http://www.w3.org/ns/shacl#datatype
sh:datatype rdf:type owl:AnnotationProperty .

###  http://www.w3.org/ns/shacl#minCount
sh:minCount rdf:type owl:AnnotationProperty .

###  http://www.w3.org/ns/shacl#targetClass
sh:targetClass rdf:type owl:AnnotationProperty .

###  http://www.w3.org/ns/shacl/dash#uniqueValueForClass
<http://www.w3.org/ns/shacl/dash#uniqueValueForClass> rdf:type owl:
    AnnotationProperty .

.....

#####
#      Datatypes
#####

###  http://www.w3.org/2001/XMLSchema#date
xsd:date rdf:type rdfs:Datatype .

#####
#      Object Properties
#####

###  http://example.org/artstate
ex:artstate rdf:type owl:ObjectProperty ;
            rdfs:domain ex:Art ;
            rdfs:range ex:State ;
            rdfs:label "Art State" .

###  http://example.org/belongsToCategory
ex:belongsToCategory rdf:type owl:ObjectProperty ;
                    owl:inverseOf ex:hasCategory ;
                    rdfs:domain ex:Art ,
                                ex:Event ;
                    rdfs:range ex:ArtCategory ,
                                ex:EventCategory ;

```

```

                                rdfs:label "Belongs To Category" .

### http://example.org/eventat
ex:eventat rdf:type owl:ObjectProperty ;
            rdfs:domain ex:Location ;
            rdfs:range ex:Event ;
            rdfs:label "Event At" .

.....

#####
# Data properties
#####

### http://example.org/address
ex:address rdf:type owl:DatatypeProperty ;
            rdfs:domain ex:Location ;
            rdfs:range xsd:string ;
            rdfs:label "Address" .

### http://example.org/day
ex:day rdf:type owl:DatatypeProperty ;
        rdfs:domain ex:Calendar ;
        rdfs:range xsd:date ;
        rdfs:label "day" .

.....

#####
# Classes
#####

### http://example.org/Art
ex:Art rdf:type owl:Class ;
        owl:equivalentClass wd:Q570116 .

### http://example.org/ArtCategory
ex:ArtCategory rdf:type owl:Class ;
                rdfs:label "Art Category" .

### http://example.org/Event
ex:Event rdf:type owl:Class ;
          owl:equivalentClass wd:Q1656682 ;
          rdfs:label "Event" .

### http://www.w3.org/ns/shacl#NodeShape
sh:NodeShape rdf:type owl:Class .

### http://www.wikidata.org/entity/Q1656682
wd:Q1656682 rdf:type owl:Class .

```

```

### http://www.wikidata.org/entity/Q570116
wd:Q570116 rdf:type owl:Class .

.....

#####
#      Individuals
#####

### http://example.org/Art
ex:Art rdf:type owl:NamedIndividual ;
      owl:sameAs schema:TouristAttraction .

### https://schema.org/TouristAttraction

### http://example.org/ArtShape
ex:ArtShape rdf:type owl:NamedIndividual ,
              sh:NodeShape ;
  sh:property [ sh:datatype xsd:string ;
                sh:minCount 1 ;
                sh:path ex:artname_it
              ] ,
              [ sh:datatype xsd:string ;
                sh:maxCount 1 ;
                sh:minCount 1 ;
                sh:path ex:artclassid
              ] ;
  sh:targetClass ex:Art .

### http://example.org/EventShape
ex:EventShape rdf:type owl:NamedIndividual ,
                  sh:NodeShape ;
  sh:property [ sh:datatype xsd:string ;
                sh:minCount 1 ;
                sh:path ex:eventname_it
              ] ,
              [ sh:datatype xsd:string ;
                sh:maxCount 1 ;
                sh:minCount 1 ;
                sh:path ex:eventclassid ;
                <http://www.w3.org/ns/shacl/dash#
uniqueValueForClass> ex:Event
              ] ,
              [ sh:class ex:Location ;
                sh:minCount 1 ;
                sh:path ex:eventat
              ] ;
  sh:targetClass ex:Event .

.....

### https://schema.org/TouristAttraction
schema:TouristAttraction rdf:type owl:NamedIndividual .

```

```

### https://schema.org/TouristTrip
schema:TouristTrip rdf:type owl:NamedIndividual .

#####
# Annotations
#####

ex:Art rdfs:label "Art" .

ex:Tour rdfs:label "Tour" .

```

Listing 4.1: Full TTL file

Defining the mapping To link the PostgreSQL relational schema with the RDF triples in the TTL file, a mapping file is used. This file defines how the relational tables and columns are translated into RDF triples, providing the necessary semantic representation for querying and analysis. The mapping file follows the R2RML (Relational to RDF Mapping Language) standard, specifying the translation rules for each table and column.

```

[PrefixDeclaration]
ex:          http://example.org/
sh:          http://www.w3.org/ns/shacl#
wd:          http://www.wikidata.org/entity/
geo:         http://www.opengis.net/ont/geosparql#
owl:         http://www.w3.org/2002/07/owl#
rdf:         http://www.w3.org/1999/02/22-rdf-syntax-ns#
xml:         http://www.w3.org/XML/1998/namespace
xsd:         http://www.w3.org/2001/XMLSchema#
obda:        https://w3id.org/obda/vocabulary#
rdfs:        http://www.w3.org/2000/01/rdf-schema#
schema:      https://schema.org/
geonames:    http://www.geonames.org/ontology#

[MappingDeclaration] @collection [[

mappingId      art-mapping
target         ex:art/{classid} ex:artclassid {classid}^^xsd:string ;
               ex:artname_it {name_it}^^xsd:string ; ex:artdescr_it {descr_it}^^
               xsd:string ; ex:artimage_url {image_url}^^xsd:string ;
               ex:area_di_download {area_di_download}^^xsd:string ; ex:open_time
               {open_time}^^xsd:string ; ex:tickets {tickets}^^xsd:string ;
               ex:artstate ex:arteventStatus/{state} ; owl:sameAs
               schema:TouristAttraction , wd:Q570116 .
source         SELECT "classid", "name_it", "descr_it", "image_url",
               "area_di_download", "open_time", "tickets", "state"
               FROM art

mappingId      event-mapping

```

```

target          ex:event/{classid} a ex:Event ; ex:eventclassid {
  classid}^^xsd:string ; ex:eventname_it {name_it}^^xsd:string ;
  ex:eventdescr_it {descr_it}^^xsd:string ; ex:eventimage_url {
    image_url}^^xsd:string ; ex:eventstate ex:arteventStatus/{state} ;
    owl:sameAs schema:Event , wd:Q1656682 .
source          SELECT "classid", "name_it", "descr_it", "image_url",
  "state"
                FROM event

mappingId        event-category-mapping
target          ex:eventCategory/{classid} a ex:EventCategory ;
  ex:eventcatclassid {classid}^^xsd:string ; ex:eventcatname_it {
    name_it}^^xsd:string .
source          SELECT "classid", "name_it"
                FROM event_category

mappingId        event-category-event-mapping
target          ex:eventCategory/{category} ex:hasCategory ex:event/{
  event} .
source          SELECT "category", "event"
                FROM a_event_category_event_category

mappingId        event-category-event-inverse-mapping
target          ex:event/{event} ex:belongsToCategory ex:eventCategory
  /{category} .
source          SELECT "category", "event"
                FROM a_event_category_event_category

mappingId        calendar-mapping
target          ex:calendar/{classid} a ex:Calendar ;
  ex:calendarclassid {classid}^^xsd:string ; ex:day {day}^^xsd:date
  ; ex:start_time {start_time}^^xsd:string ; ex:end_time {end_time
  }^^xsd:string ; ex:eventon ex:event/{event} .
source          SELECT "classid", "day", "start_time", "end_time", "
  event"
                FROM calendar

mappingId        location-mapping
target          ex:location/{classid} a ex:Location ;
  ex:locationclassid {classid}^^xsd:string ; ex:address {address}^^
  xsd:string ; ex:num {num}^^xsd:integer ; ex:geom {geom}^^
  xsd:string ; ex:eventat ex:event/{event} .
source          SELECT "classid", "address", "num", "geom", "event"
                FROM location

mappingId        event-to-location-mapping
target          ex:event/{event} a ex:Event ; ex:eventat ex:location/{
  classid} .
source          SELECT "event", "classid" FROM location WHERE "event"
  IS NOT NULL;

mappingId        event-to-calendar-mapping
target          ex:event/{event} a ex:Event ; ex:eventon ex:calendar/{
  classid} .
source          SELECT "event", "classid" FROM calendar WHERE "event"
  IS NOT NULL;

```

```
.....  
]]
```

Listing 4.2: OBDA Mapping File

For simplicity, this part focuses only on the event-related entities: EVENT, CATEGORY, LOCATION, STATUS AND CALENDAR which have been extracted and translated into a corresponding TTL file.

```
@prefix ex: <http://example.org/> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix xml: <http://www.w3.org/XML/1998/namespace> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .  
@prefix obda: <https://w3id.org/obda/vocabulary#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
  
### http://example.org/eventat  
ex:eventat rdf:type owl:ObjectProperty ;  
           rdfs:domain ex:Location ;  
           rdfs:range ex:Event ;  
           rdfs:label "Event At" .  
  
### http://example.org/eventon  
ex:eventon rdf:type owl:ObjectProperty ;  
           rdfs:domain ex:Calendar ;  
           rdfs:range ex:Event ;  
           rdfs:label "Event On" .  
  
### http://example.org/eventcatclassid  
ex:eventcatclassid rdf:type owl:DatatypeProperty ;  
                  rdfs:domain ex:EventCategory ;  
                  rdfs:range xsd:string ;  
                  rdfs:label "Eventcat Classid" .  
  
### http://example.org/eventcatname_it  
ex:eventcatname_it rdf:type owl:DatatypeProperty ;  
                  rdfs:domain ex:EventCategory ;  
                  rdfs:range xsd:string ;  
                  rdfs:label "Eventcat Name" .  
  
### http://example.org/eventclassid  
ex:eventclassid rdf:type owl:DatatypeProperty ;  
               rdfs:domain ex:Event ;  
               rdfs:range xsd:string ;  
               rdfs:label "Event Class ID" .
```

```
### http://example.org/eventdescr_it
ex:eventdescr_it rdf:type owl:DatatypeProperty ;
                  rdfs:domain ex:Event ;
                  rdfs:range xsd:string ;
                  rdfs:label "Event Description (IT)" .

### http://example.org/eventimage_url
ex:eventimage_url rdf:type owl:DatatypeProperty ;
                  rdfs:domain ex:Event ;
                  rdfs:range xsd:string ;
                  rdfs:label "Event Image URL" .

### http://example.org/eventname_it
ex:eventname_it rdf:type owl:DatatypeProperty ;
                rdfs:domain ex:Event ;
                rdfs:range xsd:string ;
                rdfs:label "Event Name" .

### http://example.org/Event
ex:Event rdf:type owl:Class ;
         rdfs:label "Event" .

### http://example.org/calendarclassid
ex:calendarclassid rdf:type owl:DatatypeProperty ;
                   rdfs:domain ex:Calendar ;
                   rdfs:range xsd:string ;
                   rdfs:label "Calendar Class ID" .

### http://example.org/day
ex:day rdf:type owl:DatatypeProperty ;
       rdfs:domain ex:Calendar ;
       rdfs:range xsd:date ;
       rdfs:label "day" .

### http://example.org/end_time
ex:end_time rdf:type owl:DatatypeProperty ;
            rdfs:domain ex:Calendar ;
            rdfs:range xsd:string ;
            rdfs:label "EndTime" .

### http://example.org/start_time
ex:start_time rdf:type owl:DatatypeProperty ;
              rdfs:domain ex:Calendar ;
              rdfs:range xsd:string ;
              rdfs:label "StartTime" .
```

```

### http://example.org/Calendar
ex:Calendar rdf:type owl:Class ;
            rdfs:label "Calendar" .

### http://example.org/Location
ex:Location rdf:type owl:Class ;
            rdfs:label "Location" .

### http://example.org/EventCategory
ex:EventCategory rdf:type owl:Class ;
                rdfs:label "Event Category" .

### http://example.org/locationclassid
ex:locationclassid rdf:type owl:DatatypeProperty ;
                  rdfs:domain ex:Location ;
                  rdfs:range xsd:string ;
                  rdfs:label "Location Class ID" .

### http://example.org/address
ex:address rdf:type owl:DatatypeProperty ;
          rdfs:domain ex:Location ;
          rdfs:range xsd:string ;
          rdfs:label "Address" .

### http://example.org/eventstate
ex:eventstate rdf:type owl:ObjectProperty ;
              rdfs:domain ex:Event ;
              rdfs:range ex:State ;
              rdfs:label "Event State" .

### http://example.org/hasCategory
ex:hasCategory rdf:type owl:ObjectProperty ;
              rdfs:domain ex:EventCategory ;
              rdfs:range ex:Event ;
              rdfs:label "Has Category" .

### http://example.org/statecode
ex:statecode rdf:type owl:DatatypeProperty ;
            rdfs:domain ex:State ;
            rdfs:range xsd:string ;
            rdfs:label "State code" .

### http://example.org/statename
ex:statename rdf:type owl:DatatypeProperty ;
            rdfs:domain ex:State ;
            rdfs:range xsd:string ;
            rdfs:label "State name" .

```

```

### http://example.org/State
ex:State rdf:type owl:Class ;
        rdfs:label "State" .

```

Listing 4.3: TTL file related to Events

Figure 4.2 shows the graph-based representation of the TTL structure in Listing 4.3.

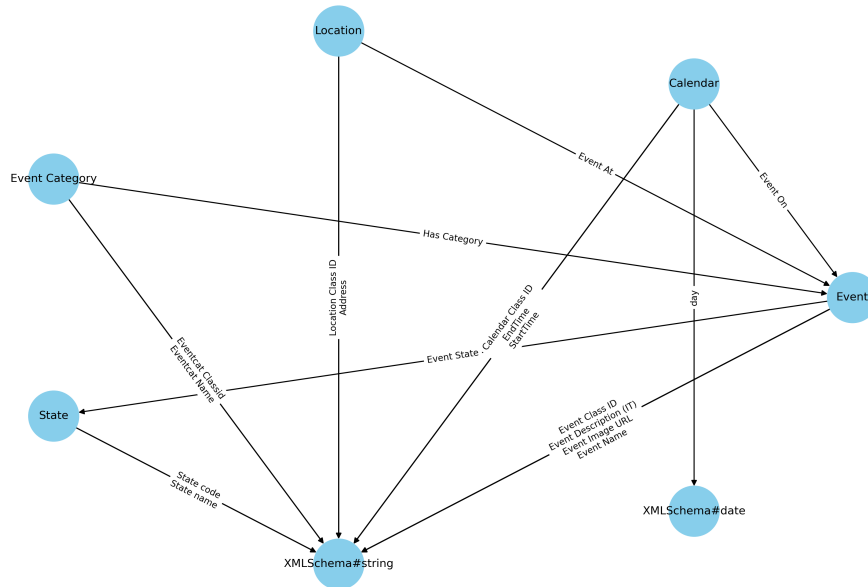


Figure 4.2: Graphical representation of event related entities and relations

The M-Schema representation in Listing 4.4 is related to the tables containing the Events and is used to increase the accuracy, through short examples, in translating NL queries about events into SPARQL queries. For example, some sentences help to specify some typical event categories (e.g., Art and Culture).

```

[Ontology] events      Schema

@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix obda: <https://w3id.org/obda/vocabulary#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

```

```

# Classes
[Classes]
ex:Event      Class      Represents an event
ex:Calendar   Class      Represents a calendar entry
ex:Location   Class      Represents a location
ex:State      Class      Represents the state of an event
ex:EventCategory Class    Represents a category of an event

# Object Properties
[Relationships]
ex:eventat    ObjectProperty (ex:Location ex:Event)
- "Event is connected to Location entity. Each event must have a
  location"
- Example:
  ex:location ex:eventat ex:event .

ex:eventon    ObjectProperty (ex:Event ex:Calendar)
- "Event is connected to Calendar entity."
- Example:
  ex:calendar ex:eventon ex:event .

ex:eventstate ObjectProperty (ex:Event ex:State)
- "An event has a status/state."

ex:belongsToCategory ObjectProperty (ex:Event ex:
  EventCategory)
- "An event belongs to a category."
- Example:
  ex:event ex:belongsToCategory ex:eventcategory .

# Datatype Properties
[Attributes]
ex:eventclassid DatatypeProperty (ex:Event xsd:string)
- "Unique identifier for an event."

ex:eventname_it DatatypeProperty (ex:Event xsd:string)
- "Event name in Italian. Each event must have a name"

ex:eventdescr_it DatatypeProperty (ex:Event xsd:string)
- "Event description in Italian."

ex:eventimage_url DatatypeProperty (ex:Event xsd:string)
- "URL of the event image."

ex:calendarclassid DatatypeProperty (ex:Calendar xsd:string)
- "Unique identifier for a calendar entry."

ex:day DatatypeProperty (ex:Calendar xsd:date)
- "Date of the event."

ex:start_time DatatypeProperty (ex:Calendar xsd:string)
- "Event start time. Time format must be 00:00 "

ex:end_time DatatypeProperty (ex:Calendar xsd:string)
- "Event end time. Time format must be 00:00 "

```

```

ex:locationclassid      DatatypeProperty      (ex:Location      xsd:string)
  - "Unique identifier for a location."

ex:address              DatatypeProperty      (ex:Location      xsd:string)
  - "Address of the location."

ex:statecode            DatatypeProperty      (ex:State        xsd:string)
  - "Code representing the state of an event."

ex:statename            DatatypeProperty      (ex:State        xsd:string)
  - "Name of the event state."
  - Example: ["eliminato", "attivo"]

ex:eventcatclassid      DatatypeProperty      (ex:EventCategory  xsd:
  string)
  - "Unique identifier for an event category."

ex:eventcatname_it      DatatypeProperty      (ex:EventCategory  xsd:
  string)
  - "Category name in Italian."
  - Example: ["Anniversari e Commemorazioni","Arte e Cultura","
  Conferenze","Incontri Convegni Congressi","Mostre","Rassegna","
  Solidariet ",
  "Visite guidate","Attivit per bambini","Campi estivi
  ragazzi","Concerti","Feste","Festival","Fiere Mercati e Sagre","
  Manifestazioni sportive",
  "Spettacoli","Spettacoli extra lirica in Arena","
  Turismo","Altro"]

```

Listing 4.4: M-schema about Events

Chapter 5

Translating NL to SparQL with LLMs

In this Chapter, we will report the methodology we followed to translate NL expressions into SparQL queries that are then executed by the OnTop framework used as a Polyglot engine. Ontop is an open source platform introduced in [6] to query databases as Virtual RDF Graphs using SPARQL.

5.1 M-Schema vs Shapes

This section focuses on the role of Shapes and M-schema in improving the accuracy and efficiency of translating natural language (NL) queries into SPARQL queries using Large Language Models (LLMs). This tries to evaluate how well LLMs can generate optimized and accurate SPARQL queries when provided with different forms of structured guidance. To experiment with this potential, *Together.ai playground* with **Llama 70B Instruct Turbo** model (fine-tuned for instruction tasks) is used.

As the baseline experiment (LLM Without Shapes or M-schema), initially, the LLM is prompted with **the TTL file related to Events depicted in *Listing 4.3*, without including Shapes or M-schema**. The structure of the prompt is:

Task: Convert a natural language question into a SPARQL query based on the given RDF data (TTL file). Please consider that all the data inside the database that to which map this TTL inside tables are in Italian.

RDF Context (Turtle format): TTL file

User Question: "What events happen on 2015-12-12 with details?"

The model correctly generated the query with an explanation. The provided SPARQL query to the user question *"What events happen on 2015-12-12?"* is the following.

```
PREFIX ex: <http://example.org/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?event ?eventName ?eventDescription ?
       eventImage
WHERE {
  ?calendar ex:day ?date .
  FILTER(xsd:date(?date) = xsd:date
         ("2015-12-12")).
  ?calendar ex:eventon ?event .
  ?event ex:eventname_it ?eventName .
  ?event ex:eventdescr_it ?eventDescription .
  ?event ex:eventimage_url ?eventImage .
}
```

The query is successfully executed on OnTop, and Figure 5.1 includes the snapshot of the result.

Execution time: 320ms. Solution mappings returned: 12.

SPARQL results	SQL translation
event	eventName eventDescription eventimage
<http://example.org/event/36486>	"Natale Insieme al Tommasoli"^^xsd:string "<p></p> <p>Il Gruppo Promozionale Qu... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/48574>	"Pippo non lo sa (nel 1940 l'Italia canta l... "<p> Compagnia Gino Fra... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/48900>	"Il Grande Teatro 2015 - 2016"^^xsd:string "Il Grande Teatro ... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/48944>	"Tamara de Lempicka"^^xsd:string "<p>Dal 20 settembre 2015 al 31 ... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/49059>	"Scandalo"^^xsd:string "<p> <p> <p>Il terzo spettacolo de... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/49235>	"Dario Fo dipinge Maria Callas"^^xsd:string "<p>Dal 22 maggio 2015 e... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/49749>	"Autunno in cantina"^^xsd:string "<p>La rassegna di teatro e musica ... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/49886>	"Aspettando Santa Lucia"^^xsd:string "<p>Sabato 12 dice... "http://portale.comune.verona.it/media/_/...
<http://example.org/event/49932>	"Viaggio alla scoperta del libro"^^xsd:string "<p>Mostra V... "http://portale.comune.verona.it/media/_B...

Figure 5.1: Model-generated query result

The chosen model successfully generates SPARQL queries but demonstrates a significant limitation: **the LLM lacks precise knowledge of the actual data in the database.** As a result, it often makes assumptions based on generic knowledge rather than actual data. For example, as shown in the SPARQL query below, provided by LLM to the user question *"What are the events for kids? Can you give me the details about those events, day, time, and location?"*, the model tries to filter by event name without identifying it as a separate category. The translation is wrong, since when executed by OnTop, it gives an empty output.

```
PREFIX ex: <http://example.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-
        schema#>

SELECT ?event ?eventName ?day ?startTime ?
        endTime ?location ?address
WHERE {
    ?event a ex:Event .
    ?event ex:eventname_it ?eventName .
    FILTER (CONTAINS(LCASE(?eventName), "bambini
        ") || CONTAINS(LCASE(?eventName), "ragazzi
        ")) .
    ?event ex:eventon ?calendar .
    ?calendar ex:day ?day .
    ?calendar ex:start_time ?startTime .
    ?calendar ex:end_time ?endTime .
    ?event ex:eventat ?location .
    ?location ex:address ?address .
}
```

By giving a more detailed query as *"What are the events that belong to the event categories related to kids? Can you give me the details about those events, day, time, and location?"*, the model provides the accurate query as follows:

```
PREFIX ex: <http://example.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-
        schema#>

SELECT ?event ?eventName ?day ?startTime ?
        endTime ?location ?address ?
        eventCategoryName
WHERE {
    ?event a ex:Event .
    ?event ex:eventname_it ?eventName .
    ?event ex:hasCategory ?eventCategory .
    ?eventCategory ex:eventcatname_it ?
        eventCategoryName .
    FILTER (CONTAINS(LCASE(?eventCategoryName), "
        bambini") || CONTAINS(LCASE(?
        eventCategoryName), "ragazzi")) .
    ?event ex:eventon ?calendar .
    ?calendar ex:day ?day .
    ?calendar ex:start_time ?startTime .
    ?calendar ex:end_time ?endTime .
    ?event ex:eventat ?location .
    ?location ex:address ?address .
}
```

As another example, we show what LLM provides as translation for the question: *"Can you retrieve events belonging to operatic shows in Arena?"*. The LLM incorrectly guesses **"opera"** as a substring in the *event category name* and **"Arena"** as in the *location*, rather than recognizing it as a distinct event category. So, the model suggested the following wrong query, which has an empty result.

```
SELECT ?event ?eventName ?eventDesc
WHERE {
    ?event a ex:Event ;
           ex:eventname_it ?eventName ;
           ex:eventdescr_it ?eventDesc ;
           ex:belongsToCategory ?eventCategory .
    ?eventCategory ex:eventcatname_it ?
        eventCategoryName .
    FILTER (CONTAINS(LCASE(?eventCategoryName), "
        opera")) .
    ?event ex:eventat ?location .
    ?location ex:address ?address .
    FILTER (CONTAINS(LCASE(?address), "Arena")) .
}
```

This highlights the usefulness of additional structured context to guide the query generation process.

Next, the LLM is prompted with the TTL file related to events with Shapes. This tries to observe how Shapes-based constraints influence query generation: The event shape includes a constraint that **all events must have a location**. For the question: *"Can you list events without a location?"*, the LLM provides the answer in Figure 5.2.

With the added shape **ex:EventShape**, we can see that it defines a constraint that every event must have at least one location associated with it, using the **ex:eventat** property.

In this case, it's not possible for an event to not have a location, because the shape **ex:EventShape** ensures that every event has at least one location.

However, if we want to query for events that do not have a location, we can still try to do so, but we should expect an empty result set, because the shape constraint ensures that every event has a location.

Here's the same query as before:

Download CodeCopy Code

PREFIX ex: <http://example.org/>

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

```
SELECT ?event ?eventName|
WHERE {
  ?event a ex:Event .
  ?event ex:eventname_it ?eventName .
  FILTER NOT EXISTS {
    ?location ex:hasEvent ?event .
  }
}
```

This query should return an empty result set, because the shape **ex:EventShape** ensures that every event has at least one location associated with it.

Figure 5.2: Model's answer highlighting the EventShape

Since **ex:EventShape** enforces the rule that every event must have a location, the query logically cannot return results. *The LLM recognizes the constraint but still suggests the query*, mentioning that the result is an empty set.

Next, to test how the model does query corrections and uses Shapes, the LLM is provided with the user question: *"I want to know the list of events so I try this query:"* `SELECT ?event ?name WHERE { ?event a ex:Event . OPTIONAL { ?event ex:eventname_it ?name . } }`

The LLM provides the answer as in Figure 5.3, by **optimizing the query** and **removing unnecessary OPTIONAL keywords and joins**.

Your query is a good start, but since `ex:eventname_it` is a mandatory property for events (as defined in the `ex:EventShape` shape), the `OPTIONAL` clause is not necessary.

Here's a simplified version of your query:

[Download Code](#)[Copy Code](#)

```
SELECT ?event ?name
WHERE {
  ?event a ex:Event ;
        ex:eventname_it ?name .
}
```

This query will return all events, along with their names, because every event is required to have a name.

Note that the `OPTIONAL` clause is used to include results that may not have a certain property, but in this case, we know that every event has a name, so we can use a simple property path instead.

Figure 5.3: Model’s answer correcting and optimizing the query

SHACL constraints help the LLM to optimize queries by removing unnecessary `OPTIONAL` clauses [22]. The LLM understands structural constraints but requires explicit instruction to leverage them effectively.

As the next step, the LLM is prompted with the **M-schema in Listing 4.4** with the same prompt structure. This significantly improved the performance of the model: for example, the LLM correctly recognized that “for kids”, there are predefined event categories, and applied the appropriate filters as below. The inclusion of example queries helps the model infer correct query structures **without applying long unnecessary filters and the model avoided guessing**, leading to more accurate and contextually relevant queries.

```
PREFIX ex: <http://example.org/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?event ?eventname ?eventdescr ?day ?
       start_time ?end_time ?location ?address
WHERE {
  ?event a ex:Event .
  ?event ex:belongsToCategory ?category .
  ?category ex:eventcatname_it ?catname .
  FILTER (?catname = "Attivit per bambini" ||
         ?catname = "Campi estivi ragazzi") .
  ?event ex:eventname_it ?eventname .
  ?event ex:eventdescr_it ?eventdescr .
  ?calendar ex:eventon ?event .
  ?calendar ex:day ?day .
  ?calendar ex:start_time ?start_time .
  ?calendar ex:end_time ?end_time .
  ?location ex:eventat ?event .
  ?location ex:address ?address .
}
```

In **M-schema**, the constraints are included as **natural language descriptions**; the model **still managed to optimize** the query—provided that **instructions explicitly asked the LLM to read descriptions carefully**, the answer given by model to the user question *Can you give me only the event names of the events that have a location?* is optimized as in figure 5.4.

Based on the description provided, "Event is connected to Location entity. Each event must have a location", we can infer that every event has a location. This means that the **ex:eventat** property is a mandatory property for every event.

With this information, we can optimize the query further by removing the **?event** **ex:eventat** **?location** triple pattern altogether. Since every event has a location, we can simply retrieve the event names without checking for the existence of a location.

Here is the optimized query:

[Download Code](#)[Copy Code](#)

```
PREFIX ex: <http://example.org/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

```
SELECT ?eventname
WHERE {
  ?event ex:eventname_it ?eventname .
}
```

This query is more efficient because it only retrieves the event names, without checking for the existence of a location. This is possible because we know that every event has a location, based on the description provided.

By reading the descriptions included in the M-schema, we can gain a deeper understanding of the data model and optimize our queries accordingly. In this case, the description provided allowed us to simplify the query and improve its performance.

Figure 5.4: Model's answer optimizing queries when provided M-schema with explicit instructions.

To conclude, Table 5.1 compares the effect of M-schema and Shapes on improving the performance of chosen LLMs.

Feature	M-schema	Shapes
Accuracy	High - Provides explicit data descriptions and examples, reducing ambiguity.	Moderate - Constraints guide query structure but do not describe data explicitly.
Query Optimization	Possible - If explicitly instructed, M-schema can guide query efficiency.	High - Built-in constraints lead to optimized SPARQL queries.
Ease of Use	High - Provides natural language descriptions, making it intuitive for users.	Moderate - Requires knowledge of Shape constraints.

Table 5.1: Feature Comparison: M-schema vs. Shapes

While **M-schema** proved to have advantageous, a **new challenge emerged**: when the prompt was **too long and complex**, the LLM struggled to process all the information effectively. To avoid this, a potential **future direction** is to implement **Retrieval-Augmented Generation (RAG)**, which dynamically retrieves relevant context instead of relying on

a single long prompt. This could improve scalability and accuracy.

5.2 Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

This section experiments on optimizing ontology-based SPARQL generation using lightweight large language models. Contrary to the previous experiments in *Section 6.1*, where a larger Llama 70B Instruct model was used, this experiment uses *Llama-2-7B Chat, quantized to 8-bit precision (Q8_0 format)*. The parameter size 7B is much smaller, which holds notable constraints, especially **the input tokens allow the prompt to be limited** and **the reasoning capacity is low**. The issue in a practical scenario is that the huge models trained on large parameter sizes often have strict usage quotas. As demonstrated in *Section 6.1*, although the Llama 70B model exhibited noteworthy performance in understanding complex ontology contexts (both M-Schema and SHACL shapes), the challenge occurs when these high-parameter models are unavailable or unaffordable. Therefore, many users try to work with free and affordable models. But these models demonstrate the limitations that they have a limited number of tokens that allow the prompt (reduced context window) and are likely to hallucinate, duplicate information, or miss ontology semantics. For this reason, this work investigates a multi-step prompting approach that ensures to provide relevant ontology context within the token limit.

The investigating approach uses **dictionary documents** as in *Listing 6.1* that mimic the TTL file in *Listing 4.3*. This is separated into two dictionaries: one for classes and one for properties, as in *Listing 6.2* and *Listing 6.3* respectively (only a part of the property dictionary is included).

```
ontology_dict = {
    "ex:eventat": '''rdf:type owl:ObjectProperty ;
    rdfs:domain ex:Location ;
    rdfs:range ex:Event ;
    rdfs:label "Event At" .''' ,

    "ex:eventon": '''rdf:type owl:ObjectProperty ;
    rdfs:domain ex:Calendar ;
    rdfs:range ex:Event ;
    rdfs:label "Event On" .''' ,

    "ex:eventcatclassid": '''rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:EventCategory ;
    rdfs:range xsd:string ;
    rdfs:label "Eventcat Classid" .''' ,
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
"ex:eventcatname_it": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:EventCategory ;
rdfs:range xsd:string ;
rdfs:label "Eventcat Name" .''',

"ex:eventclassid": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Event ;
rdfs:range xsd:string ;
rdfs:label "Event Class ID" .''',

"ex:eventdescr_it": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Event ;
rdfs:range xsd:string ;
rdfs:label "Event Description (IT)" .''',

"ex:eventimage_url": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Event ;
rdfs:range xsd:string ;
rdfs:label "Event Image URL" .''',

"ex:eventname_it": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Event ;
rdfs:range xsd:string ;
rdfs:label "Event Name" .''',

"ex:Event": '''rdf:type owl:Class ;
rdfs:label "Event" .''',

"ex:calendarclassid": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Calendar ;
rdfs:range xsd:string ;
rdfs:label "Calendar Class ID" .''',

"ex:day": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Calendar ;
rdfs:range xsd:date ;
rdfs:label "day" .''',

"ex:end_time": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Calendar ;
rdfs:range xsd:string ;
rdfs:label "EndTime" .''',

"ex:start_time": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Calendar ;
rdfs:range xsd:string ;
rdfs:label "StartTime" .''',

"ex:Calendar": '''rdf:type owl:Class ;
rdfs:label "Calendar" .''',

"ex:Location": '''rdf:type owl:Class ;
rdfs:label "Location" .''',

"ex:locationclassid": '''rdf:type owl:DatatypeProperty ;
rdfs:domain ex:Location ;
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
    rdfs:range xsd:string ;
    rdfs:label "Location Class ID" .''' ,

    "ex:address": '''rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:Location ;
    rdfs:range xsd:string ;
    rdfs:label "Address" .''' ,

    "ex:eventstate": '''rdf:type owl:ObjectProperty ;
    rdfs:domain ex:Event ;
    rdfs:range ex:State ;
    rdfs:label "Event State" .''' ,

    "ex:hasCategory": '''rdf:type owl:ObjectProperty ;
    rdfs:domain ex:ArtCategory ,
                      ex:EventCategory ;
    rdfs:range ex:Art ,
              ex:Event ;
    rdfs:label "Has Category" .''' ,

    "ex:statecode": '''rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:State ;
    rdfs:range xsd:string ;
    rdfs:label "State code" .''' ,

    "ex:statename": '''rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:State ;
    rdfs:range xsd:string ;
    rdfs:label "State name" .''' ,

    "ex:State": '''rdf:type owl:Class ;
    rdfs:label "State" .'''
}
```

Listing 5.1: Dictionary documents creation for TTL file

```
{'ex:Event': 'rdf:type owl:Class ;\n    rdfs:label "Event" .',
'ex:Calendar': 'rdf:type owl:Class ;\n    rdfs:label "Calendar" .',
'ex:Location': 'rdf:type owl:Class ;\n    rdfs:label "Location" .',
'ex:EventCategory': 'rdf:type owl:Class ;\n    rdfs:label "Event\n    Category" .',
'ex:State': 'rdf:type owl:Class ;\n    rdfs:label "State" .'}
```

Listing 5.2: Dictionary created for classes

```
{'ex:eventat': 'rdf:type owl:ObjectProperty ;\n    rdfs:domain ex:\n    Location ;\n    rdfs:range ex:Event ;\n    rdfs:label "Event At"\n    .',
'ex:eventon': 'rdf:type owl:ObjectProperty ;\n    rdfs:domain ex:\n    Calendar ;\n    rdfs:range ex:Event ;\n    rdfs:label "Event On"\n    .',
'ex:eventcatclassid': 'rdf:type owl:DatatypeProperty ;\n    rdfs:\n    domain ex:EventCategory ;\n    rdfs:range xsd:string ;\n    rdfs:\n    label "Eventcat Classid" .',
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
'ex:eventcatname_it': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:EventCategory ;\n    rdfs:range xsd:string ;\n    rdfs:label "Eventcat Name" .',
'ex:eventclassid': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Class ID" .',
'ex:eventdescr_it': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Description (IT)" .',
'ex:eventimage_url': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Image URL" .',
'ex:eventname_it': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Name" .',
'ex:calendarclassid': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:string ;\n    rdfs:label "Calendar Class ID" .',
'ex:day': 'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:date ;\n    rdfs:label "day" .',
```

Listing 5.3: A sample of the property dictionary

To evaluate the performance of the model, the experiments include **five sample user questions**:

- What are the names of the events happening on 2015-12-12?
- What are the names and addresses of the events belong to 'Kids' category?
- What are the locations of the events happen on 2015-12-12?
- What are the events scheduled to start at 11.30?
- Can you give the event names and their status?

Provided the NL user question, the method includes two stages:

Class Filtering: A prompt as follows is constructed by giving as input the class dictionary as a loop asking whether each class is relevant or not to answer the user's question. The response includes a filtered list of classes that the model identified as relevant to answer the question.

```
prompt = f"""\nYou are a SPARQL expert. Given the user question: "{question}"\nIs the following class relevant? {class_}\nAnswer only YES or NO.""
```

The identified relevant classes are stored in a list:

```
{'question': 'What are the names of the events happening on\n2015-12-12?',\n 'response': [('ex:EventCategory',
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```

    'rdf:type owl:Class ;\n      rdfs:label "Event Category" .')]],
{'question': "What are the names and addresses of the events belong
to 'Kids' category?",
 'response': [('ex:Location',
 'rdf:type owl:Class ;\n      rdfs:label "Location" .'),
 ('ex:EventCategory',
 'rdf:type owl:Class ;\n      rdfs:label "Event Category" .')]],

{'question': 'What are the locations of the events happen on
2015-12-12?',
 'response': [('ex:Event', 'rdf:type owl:Class ;\n      rdfs:label "
Event" .'),
 ('ex:Calendar', 'rdf:type owl:Class ;\n      rdfs:label "Calendar"
.')]},

{'question': 'What are the events scheduled to start at 11.30?',
 'response': [('ex:Event', 'rdf:type owl:Class ;\n      rdfs:label "
Event" .'),
 ('ex:Calendar', 'rdf:type owl:Class ;\n      rdfs:label "Calendar"
.'),
 ('ex:EventCategory',
 'rdf:type owl:Class ;\n      rdfs:label "Event Category" .')]],

{'question': 'Can you give the event names and their status?',
 'response': [('ex:Event', 'rdf:type owl:Class ;\n      rdfs:label "
Event" .'),
 ('ex:Calendar', 'rdf:type owl:Class ;\n      rdfs:label "Calendar"
.')]}}

```

While the model correctly identifies some relevant ontology classes, it makes key mistakes in each case, either by missing important classes or including irrelevant ones. Table 5.2 includes the evaluation of the performance of the model in the first stage. For each row, the accuracy is calculated as follows.

$$\text{Accuracy (\%)} = \frac{\text{Number of Correct Matches}}{\text{Number of Expected Classes}} \times 100$$

Example:

$$\text{Accuracy} = \frac{2}{3} \times 100 = 66.7\%$$

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

No.	Question	Expected Classes	Predicted Count	Correct	Missing	Extra	Accuracy (%)
1	What are the names of the events happening on 2015-12-12?	Event, Calendar	1	0	1	1	0
2	Names and addresses of the events in 'Kids' category?	Event, Location, EventCategory	2	2	1	0	66.7
3	Locations of the events on 2015-12-12?	Event, Calendar, Location	2	2	1	0	66.7
4	Events scheduled to start at 11.30?	Event, Calendar	3	2	0	1	66.7
5	Event names and their status?	Event, Status	2	1	1	1	50

Table 5.2: Numerical Evaluation of Class Filtering Results

Average Accuracy:

$$\text{Average Accuracy} = \frac{0 + 66.7 + 66.7 + 66.7 + 50}{5} = 50\%$$

Property Filtering: Given the identified classes and the user question, a prompt is constructed to loop through the property dictionary and extract the relevant properties. The prompt is structured as follows:

```
prompt = f"""You are a SPARQL expert.
Given the user question: "{question}" and the class {class_name} with
description:
{class_description}

Is the following property relevant for retrieving information from
class {class_name}?

{prop_name} {prop_description}

Answer only YES or NO."""
```

The properties extracted by the model are stored in a list. For example, the answer to the first question is as below:

```
{'question': 'What are the names of the events happening on
2015-12-12?',

'relevant_classes': {'ex:EventCategory': 'rdf:type
owl:Class ;\n      rdfs:label "Event Category" .'},

'response': {'ex:EventCategory': [('ex:eventat',
'rdf:type owl:ObjectProperty ;\n      rdfs:domain ex:Location ;\n      rdfs:range ex:Event ;\n      rdfs:label "Event At" .'),

('ex:eventcatclassid',
'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:
EventCategory ;\n      rdfs:range xsd:string ;\n      rdfs:label "
Eventcat Classid" .'),

('ex:eventclassid',
'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Event ;\n      rdfs:range xsd:string ;\n      rdfs:label "Event Class ID" .)],
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
('ex:day',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Calendar ;\n      rdfs:range xsd:date ;\n      rdfs:label "day" .'),

('ex:end_time',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Calendar ;\n      rdfs:range xsd:string ;\n      rdfs:label "EndTime" .'),

('ex:start_time',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Calendar ;\n      rdfs:range xsd:string ;\n      rdfs:label "StartTime" .'),

('ex:eventstate',
 'rdf:type owl:ObjectProperty ;\n      rdfs:domain ex:Event ;\n      rdfs:range ex:State ;\n      rdfs:label "Event State" .'),

('ex:statecode',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:State ;\n      rdfs:range xsd:string ;\n      rdfs:label "State code" .')]]},
```

As a further step, the same prompt is input with the correct relevant classes with user question and extracts the properties suggested by the model. For example, the answer provided by model to the same question mentioned above is,

```
{'question': 'What are the names of the events happening on
2015-12-12?',

'relevant_classes': {'ex:Event': 'rdf:type owl:Class ;\n      rdfs:
label "Event" .',

'ex:Calendar': 'rdf:type owl:Class ;\n      rdfs:label "Calendar"
.'},

'response': {'ex:Event': [('ex:eventcatclassid',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:
EventCategory ;\n      rdfs:range xsd:string ;\n      rdfs:label "
Eventcat Classid" .'),

('ex:eventcatname_it',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:
EventCategory ;\n      rdfs:range xsd:string ;\n      rdfs:label "
Eventcat Name" .'),

('ex:eventclassid',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Event ;\n
rdfs:range xsd:string ;\n      rdfs:label "Event Class ID" .'),

('ex:eventimage_url',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Event ;\n
rdfs:range xsd:string ;\n      rdfs:label "Event Image URL" .'),

('ex:eventname_it',
 'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:Event ;\n
rdfs:range xsd:string ;\n      rdfs:label "Event Name" .'),
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
('ex:calendarclassid',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:string ;\n    rdfs:label "Calendar Class ID"
. '),

('ex:end_time',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:string ;\n    rdfs:label "EndTime" . '),

('ex:start_time',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:string ;\n    rdfs:label "StartTime" . '),

('ex:hasCategory',
  'rdf:type owl:ObjectProperty ;\n    rdfs:domain ex:EventCategory
;\n    rdfs:range ex:Event ;\n    rdfs:label "Has Category" . '),

('ex:statecode',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:State ;\n    rdfs:range xsd:string ;\n    rdfs:label "State code" . '),

('ex:statename',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:State ;\n    rdfs:range xsd:string ;\n    rdfs:label "State name" . ')],

'ex:Calendar': [('ex:eventat',
  'rdf:type owl:ObjectProperty ;\n    rdfs:domain ex:Location ;\n    rdfs:range ex:Event ;\n    rdfs:label "Event At" . '),

('ex:eventimage_url',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Image URL" . '),

('ex:eventname_it',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Event ;\n    rdfs:range xsd:string ;\n    rdfs:label "Event Name" . '),

('ex:calendarclassid',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:string ;\n    rdfs:label "Calendar Class ID"
. '),

('ex:day',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Calendar ;\n    rdfs:range xsd:date ;\n    rdfs:label "day" . '),

('ex:address',
  'rdf:type owl:DatatypeProperty ;\n    rdfs:domain ex:Location ;\n    rdfs:range xsd:string ;\n    rdfs:label "Address" . '),

('ex:hasCategory',
  'rdf:type owl:ObjectProperty ;\n    rdfs:domain ex:EventCategory
;\n    rdfs:range ex:Event ;\n    rdfs:label "Has Category" . '),

('ex:statecode',
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
'rdf:type owl:DatatypeProperty ;\n      rdfs:domain ex:State ;\n      rdfs:range xsd:string ;\n      rdfs:label "State code" .')]]},
```

As demonstrated in above example outputs, when extracting properties with this approach, it results *in repetition and irrelevant property suggestions*, as the prompt still includes properties that are semantically unrelated to the question or class context. For instance, in first output, **ex:eventat** property is extracted as a related property to **ex:EventCategory** class and in second output as a related property to **ex:Calendar**. But *ex:eventat* builds the relationship between **ex:Event** and **ex:Location** classes.

To address these issues of the dictionary-based approach, a **graph-based traversal** method is investigated. This is used to extract only those properties that are connected to the given classes. This reduces the number of tokens while avoiding repetition and providing a prompt with more relevant properties to mitigate unrelated property suggestions.

The process consists of the following steps:

As initial step, the TTL file is parsed into a directed multigraph using *rdflib* and *networkx* libraries. Each class is presented as a graph node, and each property becomes an edge between class nodes. Object properties form edges between class nodes, while datatype properties link class nodes to literals. Figure 4.2 shows the relevant graphical illustration.

After identifying relevant classes from prompt-based filtering, a function is used to travel through the ontology graph starting from each relevant class node. This minimizes the set of properties to those closely related to the relevant class. This adopts a *depth-limited traversal* (up to a specified number of hops, default is 2) using **breadth-first search (BFS)**. It collects all properties (edges) that are reachable from the starting class node within this depth limit. This process uses a queue to manage nodes during traversal, storing both the node and current depth. At each step, it collects outgoing edges from the current node, along with edge metadata (e.g., label and type), and avoids revisiting nodes using a visited set. The output is a list of dictionaries representing properties reachable within the allowed depth. For example, given the relevant classes *Event* and *Calendar*, the following list of properties is returned from the function.

```
[('http://example.org/Event',  
  'http://www.w3.org/2001/XMLSchema#string',  
  'Event Description (IT)',  
  'DatatypeProperty'),  
  
 ('http://example.org/Event',  
  'http://example.org/State',  
  'Event State',  
  'ObjectProperty')],
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

```
('http://example.org/Event',
 'http://www.w3.org/2001/XMLSchema#string',
 'Event Image URL',
 'DatatypeProperty'),

('http://example.org/Calendar',
 'http://www.w3.org/2001/XMLSchema#string',
 'Calendar Class ID',
 'DatatypeProperty'),

('http://example.org/State',
 'http://www.w3.org/2001/XMLSchema#string',
 'State name',
 'DatatypeProperty'),

('http://example.org/Calendar',
 'http://www.w3.org/2001/XMLSchema#string',
 'StartTime',
 'DatatypeProperty'),

('http://example.org/State',
 'http://www.w3.org/2001/XMLSchema#string',
 'State code',
 'DatatypeProperty'),

('http://example.org/Calendar',
 'http://example.org/Event',
 'Event On',
 'ObjectProperty'),

('http://example.org/Event',
 'http://www.w3.org/2001/XMLSchema#string',
 'Event Class ID',
 'DatatypeProperty'),

('http://example.org/Event',
 'http://www.w3.org/2001/XMLSchema#string',
 'Event Name',
 'DatatypeProperty'),

('http://example.org/Calendar',
 'http://www.w3.org/2001/XMLSchema#date',
 'day',
 'DatatypeProperty'),

('http://example.org/Calendar',
 'http://www.w3.org/2001/XMLSchema#string',
 'EndTime',
 'DatatypeProperty')]
```

This guarantees that only semantically close and relevant properties are considered in the next stage when providing a prompt with properties to choose which are relevant given the user’s question. For instance, this does not provide any properties related to *EventCategory*, which are irrelevant.

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

This helps the prompt reduce noise as it is provided with mostly related properties to choose from. With this step, it reduces the tendency that prompts extract properties that are semantically unrelated to the question and class context.

Before including the prompt, the relevant classes and collected properties are structured into a format similar to the representation in the TTL file. Finally, the prompt is input with the user question, relevant classes, and their nearby properties structured similarly to TTL representation and asks to extract relevant properties. The mechanism and the prompt are structured as,

```
class_defs = ""
for cls in relevant_classes:
    class_uri = cls.split("/")[-1]
    class_defs += f"ex:{class_uri} rdf:type owl:Class ;\n    rdfs:label \"{class_uri}\" .\n\n"
print(class_defs)

collected_props
prop_defs = ""
for prop in all_collected_props:
    label = prop[2]
    prop_type = prop[3]
    from_cls = prop[0].split("/")[-1]
    to_cls = prop[1].split("/")[-1]

    if prop_type.lower() == "objectproperty":
        prop_defs += (
            f"ex:{label} rdf:type owl:ObjectProperty ;\n"
            f"    rdfs:domain ex:{from_cls} ;\n"
            f"    rdfs:range ex:{to_cls} ;\n"
            f"    rdfs:label \"{label}\" .\n\n"
        )
    else:
        prop_defs += (
            f"ex:{label} rdf:type owl:DatatypeProperty ;\n"
            f"    rdfs:domain ex:{from_cls} ;\n"
            f"    rdfs:range xsd:string ;\n"
            f"    rdfs:label \"{label}\" .\n\n"
        )
print(prop_defs)

prompt = (
    f"You are a semantic web expert.\n\n"
    f"The following ontology definitions are relevant to the\n"
    f"classes:\n\n"
    f"{class_defs}"
    f"{prop_defs}"
    f"Based on these definitions, which properties are relevant\n"
    f"for answering the question:\n"
    f"'{question}'\n\n"
    f"List only the property labels, separated by commas, without\n"
    f"explanation."
```

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

)

For example, to the given user question *What are the names of the events happening on 2015-12-12?*, before input to the prompt the for loops structure the relevant classes and collected properties: `print ( class_defs )` and `print ( prop_defs )` demonstrate this (how the classes and properties are structured before including to prompt) as below,

```
ex:Event rdf:type owl:Class ;
    rdfs:label "Event" .

ex:Location rdf:type owl:Class ;
    rdfs:label "Location" .

ex:Event Class ID rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:Event ;
    rdfs:range xsd:string ;
    rdfs:label "Event Class ID" .

ex:Event Name rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:Event ;
    rdfs:range xsd:string ;
    rdfs:label "Event Name" .

ex:State code rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:State ;
    rdfs:range xsd:string ;
    rdfs:label "State code" .

ex:Event State rdf:type owl:ObjectProperty ;
    rdfs:domain ex:Event ;
    rdfs:range ex:State ;
    rdfs:label "Event State" .

ex:State name rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:State ;
    rdfs:range xsd:string ;
    rdfs:label "State name" .

ex:Event Description (IT) rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:Event ;
    rdfs:range xsd:string ;
    rdfs:label "Event Description (IT)" .

ex:Event Image URL rdf:type owl:DatatypeProperty ;
    rdfs:domain ex:Event ;
    rdfs:range xsd:string ;
    rdfs:label "Event Image URL" .
```

The evaluation of the performance of the model in identifying relevant properties is presented in Table 5.3. The model's predictions were evaluated using standard classification metrics—**Precision**, **Recall**, and **F1-**

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

Score—to more accurately capture the system’s performance.

These metrics are computed as follows:

- **Precision** = $\frac{\text{Correct}}{\text{Correct} + \text{Extra}}$
- **Recall** = $\frac{\text{Correct}}{\text{Correct} + \text{Missing}}$
- **F1-Score** = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Precision measures the fraction of predicted properties that were correct, whereas recall calculates how many of the required properties were successfully identified. F1-Score balances these two metrics to provide a single effectiveness score.

No.	Question	Predicted Properties	Total Predicted	Actual Correct	Missing	Extra	Precision (%)	Recall (%)	F1-Score (%)
1	Locations of the events on 2015-12-12?	Event Class ID, StartTime, EndTime, Event Name, Location Class ID, Address, State code, Event On, Event At, Event State, State name, Event Description (IT), Event Image URL	13	9	1	4	69.2	90.0	78.3
2	Names of the events happening on 2015-12-12?	Event Name, StartTime, EndTime, Event On, Event State, State name, Event Image URL	7	2	1	5	28.6	66.7	40.0
3	Names and addresses of the events in 'Kids' category?	Event Class ID, Event Name, Eventcat Name, State name, Event At	5	4	2	1	80.0	66.7	72.7
4	Events scheduled to start at 11:30?	Event Class ID, Event Name, StartTime, State code, Event On, Event State, Day	7	4	0	3	57.1	100.0	72.7
5	Event names and their status?	Event Class ID, Event Name, Event State, State code, State name, Event Description (IT), Event Image URL	7	5	0	2	71.4	100.0	83.3

Table 5.3: Evaluation of Property Extraction Including Precision, Recall, and F1-Score

These metrics facilitate a more reliable evaluation of extraction performance:

- **Row 1** has high recall (90%) and improved precision (69.2%), highlighting the model retrieves most relevant properties but includes some extras.
- **Row 2** shows low precision (28.6%) and moderate recall, indicating a high number of false positives.
- **Rows 4 and 5** achieve perfect recall and balanced precision, having strong F1-scores.
- **The balanced F1-scores**—ranging from 40% to 83.3%—illustrate that this approach demonstrates adequate performance in identifying relevant properties and also avoiding excessive noise, unlike dictionary-based approach which tend to include many loosely related or irrelevant terms.

5.2. Optimizing Prompt Structure for Lightweight LLMs: Llama-2-7B (Quantized) Experiments

To conclude, by making the prompt more focused, the approach shows improved performance in retrieving relevant properties, especially due to the benefits of reduced noise through providing non-repetitive and mostly closed properties to the prompt to choose from.

Chapter 6

Conclusions

This thesis explores a methodology for querying heterogeneous data sources using Virtual Knowledge Graphs (VKGs) as a global schema and Large Language Models (LLMs) for translating natural language expressions into SpaQL queries to be applied to the VKGs. It proposes a polyglot query engine that integrates diverse structured data into a unified semantic layer via RDF, Virtual Knowledge Graphs (VKGs), and Ontop for ontology-based data access (OBDA). The work leverages LLMs to translate natural language questions into SPARQL queries, incorporating SHACL for data validation and exploring structured guidance to optimize query generation. The research bridges natural language interfaces and semantic technologies, contributing both practical tools and theoretical insights. As a future work, it is possible to explore how Virtual Knowledge Graphs can support multi-modal query recommendations, starting from a natural language expression, in contexts where multiple datasets, such as text, images, and structured data, must be accessed.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, Professor Elisa Quintarelli, for her constant support, insightful guidance, and encouragement throughout my studies and the development of this thesis. Her availability and thoughtful feedback have been influential in shaping both my academic growth and this research.

My sincere thanks also go to my co-supervisor, Professor Matteo Lissandrini, whose expertise suggestions, and encouraging ideas greatly enriched the quality of this work.

Finally, I extend my appreciation to all those who, directly or indirectly, contributed to this journey with their knowledge, patience, and kindness.

Bibliography

- [1] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier *et al.*, “Knowledge graphs,” *ACM Computing Surveys (Csur)*, vol. 54, no. 4, pp. 1–37, 2021.
- [2] D. Brickley, R. V. Guha, and B. McBride, “Rdf schema 1.1,” *W3C recommendation*, vol. 25, no. 2004-2014, p. 10, 2014.
- [3] S. Shanker, “Sparql-the query language of smantic web.”
- [4] J. Pérez, M. Arenas, and C. Gutierrez, “Semantics and complexity of sparql,” in *International semantic web conference*. Springer, 2006, pp. 30–43.
- [5] G. Xiao, L. Ding, B. Cogrel, and D. Calvanese, “Virtual knowledge graphs: An overview of systems and use cases,” *Data Intelligence*, vol. 1, no. 3, pp. 201–223, 06 2019. [Online]. Available: https://doi.org/10.1162/dint_a_00011
- [6] D. Calvanese, B. Cogrel, S. Komla-Ebri, R. Kontchakov, D. Lanti, M. Rezk, M. Rodriguez-Muro, and G. Xiao, “Ontop: Answering sparql queries over relational databases,” *Semantic Web*, vol. 8, no. 3, pp. 471–487, 2016.
- [7] D. Fensel, U. Simsek, K. Angele, E. Huaman, E. Kärle, O. Panasiuk, I. Toma, J. Umbrich, and A. Wahler, *Knowledge graphs*. Springer, 2020.
- [8] R. B. Thapa and M. Giese, “Optimizing sparql queries with shacl,” in *International Semantic Web Conference*. Springer, 2023, pp. 41–60.
- [9] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, J. Gao, L. Mou, and Y. Li, “A preview of xiyen-sql:

- A multi-generator ensemble framework for text-to-sql,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.08599>
- [10] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, “A review on large language models: Architectures, applications, taxonomies, open issues and challenges,” *IEEE access*, vol. 12, pp. 26 839–26 874, 2024.
- [11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [12] H. Babaei Giglou, J. D’Souza, and S. Auer, “Llms4ol: Large language models for ontology learning,” in *International Semantic Web Conference*. Springer, 2023, pp. 408–427.
- [13] S. Shaik, P. Kanakam, S. M. Hussain, and D. Suryanarayana, “Transforming natural language query to sparql for semantic information retrieval,” *International Journal of Engineering Trends and Technology*, vol. 7, pp. 347–350, 2016.
- [14] H. Yamada, T. Suzuki, Y. Ito, and J. Nemoto, “Scalardb: Universal transaction manager for polystores,” *Proc. VLDB Endow.*, vol. 16, no. 12, p. 3768–3780, Aug. 2023. [Online]. Available: <https://doi.org/10.14778/3611540.3611563>
- [15] R. Alotaibi, B. Cautis, A. Deutsch, M. Latrache, I. Manolescu, and Y. Yang, “Estocada: towards scalable polystore systems,” *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 2949–2952, 2020.
- [16] E. G. Kalaycı, I. Grangel González, F. Lösch, G. Xiao, A. ul Mehdi, E. Kharlamov, and D. Calvanese, “Semantic integration of bosch manufacturing data using virtual knowledge graphs,” in *The Semantic Web—ISWC 2020: 19th International Semantic Web Conference, Athens, Greece, November 2–6, 2020, Proceedings, Part II 19*. Springer, 2020, pp. 464–481.
- [17] A. Helal, M. Helali, K. Ammar, and E. Mansour, “A demonstration of kglac: A data discovery and enrichment platform for data science,” *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 2675–2678, 2021.

- [18] C. V. S. Avila, V. M. Vidal, W. Franco, and M. A. Casanova, “Experiments with text-to-sparql based on chatgpt,” in *2024 IEEE 18th International Conference on Semantic Computing (ICSC)*, 2024, pp. 277–284.
- [19] L. Kovriguina, R. Teucher, D. Radyush, and D. Mouromtsev, “SPARQLGEN: one-shot prompt-based approach for SPARQL query generation,” in *Proceedings of the Posters and Demo Track of the 19th International Conference on Semantic Systems co-located with 19th International Conference on Semantic Systems (SEMANTiCS 2023), Leipzig, Germany, September 20 to 22, 2023*, ser. CEUR Workshop Proceedings, N. Keshan, S. Neumaier, A. L. Gentile, and S. Vahdati, Eds., vol. 3526. CEUR-WS.org, 2023. [Online]. Available: <https://ceur-ws.org/Vol-3526/paper-08.pdf>
- [20] C. V. S. Avila, V. M. Vidal, W. Franco, and M. A. Casanova, “Few-shot learning or rag in llm-based text-to-sparql? why not both?”
- [21] V. Emonet, J. Bolleman, S. Duvaud, T. M. de Farias, and A. C. Sima, “Llm-based sparql query generation from natural language over federated knowledge graphs,” *arXiv preprint arXiv:2410.06062*, 2024.
- [22] R. B. Thapa and M. Giese, “Optimizing sparql queries withnbsps;shacl,” in *The Semantic Web – ISWC 2023: 22nd International Semantic Web Conference, Athens, Greece, November 6–10, 2023, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 41–60. [Online]. Available: https://doi.org/10.1007/978-3-031-47240-4_3